U25AI055

Lab-3

Experiment 1: Design a Fully-Featured Singly Linked List ADT

```
// 1: Design a fully featured singly linked list ADT
#include <stdio.h>
#include <stdlib.h>
struct Node
{
	int data;
	struct Node *next;
};
struct Node *insertAtHead(struct Node *head, int element);
struct Node *insertAtTail(struct Node *head, int element);
struct Node *deleteByValue(struct Node *head, int element);
struct Node *deleteByPosition(struct Node *head, int position);
struct Node *searchForValue(struct Node *head, int element);
struct Node *reverseList(struct Node *head);
void displayList(struct Node *head);
int detectCycle(struct Node *head);
int main()
{
	int n, i, choice, element, position;
	struct Node *head, *newNode, *temp;
	printf("Enter number of nodes: ");
	scanf("%d", &n);
	if (n == 0)
	{
		head = NULL;
```

```
}
else
{
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element 1: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        head = newNode;
        temp = newNode;
        for (i = 2; i <= n; i++)
        {
                newNode = (struct Node*)malloc(sizeof(struct Node));
                printf("Enter element %d: ", i);
                scanf("%d", &newNode->data);
                newNode->next = NULL;
                temp->next = newNode;
                temp = temp->next;
        }
}
do
{
        printf("1: Insert at head\n");
        printf("2: Insert at tail\n");
        printf("3: Delete by value\n");
        printf("4: Delete by position\n");
        printf("5: Search for value\n");
        printf("6: Reverse list\n");
        printf("7: Display list\n");
```

```
printf("8: Detect cycle\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch (choice)
{
        case 1:
                printf("Enter element: ");
                scanf("%d", &element);
                head = insertAtHead(head, element);
                break;
        case 2:
                printf("Enter element: ");
                scanf("%d", &element);
                head = insertAtTail(head, element);
                break;
        case 3:
                printf("Enter element: ");
                scanf("%d", &element);
                head = deleteByValue(head, element);
                break;
        case 4:
                printf("Enter position: ");
                scanf("%d", &position);
                head = deleteByPosition(head, position);
                break;
        case 5:
                printf("Enter element: ");
                scanf("%d", &element);
```

```
                printf("Element found at %u", searchForValue(head,
element));
                break;
            case 6:
                head = reverseList(head);
                break;
            case 7:
                printf("Elements of list are ");
                displayList(head);
                printf("\n");
                break;
            case 8:
                if (detectCycle(head))
                {
                    printf("Linked list is cyclic\n");
                }
                else
                {
                    printf("Linked list is not cyclic\n");
                }
                break;
            default:
                printf("Invalid choice\n");
        }
    } while (choice >= 1 && choice <= 8);
    return 0;
}
struct Node *insertAtHead(struct Node *head, int element)
```

```
{
	struct Node *newNode;
	newNode = (struct Node*)malloc(sizeof(struct Node));
	newNode->data = element;
	newNode->next = head;
	head = newNode;
	return head;
}
struct Node *insertAtTail(struct Node *head, int element)
{
	struct Node *newNode, *ptr;
	newNode = (struct Node*)malloc(sizeof(struct Node));
	newNode->data = element;
	newNode->next = NULL;
	ptr = head;
	if (head == NULL)
	{
		head = newNode;
		return head;
	}
	else
	{
		while (ptr->next != NULL)
		{
			ptr = ptr->next;
		}
		ptr->next = newNode;
		return head;
```

```
        }
}
struct Node *deleteByValue(struct Node *head, int element)
{
        if (head == NULL)
        {
                return NULL;
        }
        struct Node *fast, *slow;
        slow = fast = head;
        while (fast->data != element && fast->next != NULL)
        {
                slow = fast;
                fast = fast->next;
        }
        if (slow == fast)
        {
                head = fast->next;
                free(fast);
        }
        if (fast->data == element)
        {
                slow->next = fast->next;
                free(fast);
        }
        return head;
}
struct Node *deleteByPosition(struct Node *head, int position)
```

```
{
        if (head == NULL)
        {
                return NULL;
        }
        int i;
        struct Node *fast, *slow;
        slow = fast = head;
        for (i = 0; i < position && fast->next != NULL; i++)
        {
                slow = fast;
                fast = fast->next;
        }
        if (slow == fast)
        {
                head = fast->next;
                free(fast);
                return head;
        }
        if (i == position)
        {
                slow->next = fast->next;
                free(fast);
                return head;
        }
}
struct Node *searchForValue(struct Node *head, int element)
{
```

```
        if (head == NULL)
        {
                return NULL;
        }
        struct Node *ptr;
        ptr = head;
        while (ptr->data != element && ptr->next != NULL)
        {
                ptr = ptr->next;
        }
        if (ptr->data == element)
        {
                return ptr;
        }
}
struct Node *reverseList(struct Node *head)
{
        struct Node *ptr, *fast, *slow;
        ptr = head;
        slow = fast = NULL;
        while (ptr != NULL)
        {
                fast = ptr;
                ptr = ptr->next;
                fast->next = slow;
                slow = fast;
        }
        head = fast;
```

```
	return head;
}
void displayList(struct Node *head)
{
	struct Node *ptr;
	ptr = head;
	while (ptr != NULL)
	{
		printf("%d\t", ptr->data);
		ptr = ptr->next;
	}
}
int detectCycle(struct Node *head)
{
	struct Node *fast, *slow;
	slow = fast = head;
	while (fast != NULL && fast->next != NULL)
	{
		slow = slow->next;
		fast = (fast->next)->next;
		if (slow == fast)
		{
			return 1;
		}
	}
	return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 12
Enter element 2: 23
Enter element 3: 34
Enter element 4: 45
Enter element 5: 56
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 1
Enter element: 67
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 7
Elements of list are 67 12      23      34      45      56
```

```
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 2
Enter element: 78
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 7
Elements of list are 67 12      23      34      45      56      78
```

```
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 3
Enter element: 34
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 7
Elements of list are 67 12      23      45      56      78
```

```
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 4
Enter position: 0
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 7
Elements of list are 12 23      45      56      78
```

```
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 5
Enter element: 56
Element found at 12915920
```

```
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 6
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 7
Elements of list are 78 56      45      23      12
```

```
1: Insert at head
2: Insert at tail
3: Delete by value
4: Delete by position
5: Search for value
6: Reverse list
7: Display list
8: Detect cycle
Enter choice: 8
Linked list is not cyclic
```

Experiment 2: Insert at Beginning with Undo Capability

```
// 2: Insert at beginning with undo capability

#include <stdio.h>

#include <stdlib.h>

struct Node

{

        int data;
```

```
	struct Node *next;
};
struct Node *insertAtBeginning(struct Node *head, int element);
struct Node *undo(const struct Node *originalHead, struct Node *head);
void displayList(struct Node *head);
int main()
{
	int n, i, choice, element;
	const struct Node *originalHead;
	struct Node *head, *newNode, *temp;
	printf("Enter number of nodes: ");
	scanf("%d", &n);
	if (n == 0)
	{
		originalHead = head = NULL;
	}
	else
	{
		newNode = (struct Node*)malloc(sizeof(struct Node));
		printf("Enter element 1: ");
		scanf("%d", &newNode->data);
		newNode->next = NULL;
		originalHead = head = newNode;
		temp = newNode;
		for (i = 2; i <= n; i++)
		{
			newNode = (struct Node*)malloc(sizeof(struct Node));
			printf("Enter element %d: ", i);
```

```
            scanf("%d", &newNode->data);
            newNode->next = NULL;
            temp->next = newNode;
            temp = temp->next;
        }
}
do
{
        printf("1. Insert at beginning\n");
        printf("2. Undo\n");
        printf("3. Display list\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &element);
                head = insertAtBeginning(head, element);
                break;
            case 2:
                head = undo(originalHead, head);
                break;
            case 3:
                printf("Elements of list are ");
                displayList(head);
                printf("\n");
                break;
```

```
                        default:
                                printf("Invalid choice\n");
                }
        } while (choice >= 1 && choice <= 3);
        return 0;
}
struct Node *insertAtBeginning(struct Node *head, int element)
{
        struct Node *newNode;
        newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = element;
        newNode->next = head;
        head = newNode;
        return head;
}
struct Node *undo(const struct Node *originalHead, struct Node *head)
{
        if (originalHead == head)
        {
                return head;
        }
        else
        {
                struct Node *temp;
                temp = head;
                head = temp->next;
                free(temp);
                return head;
```

```
        }
}
void displayList(struct Node *head)
{
        struct Node *ptr;
        ptr = head;
        while (ptr != NULL)
        {
                printf("%d\t", ptr->data);
                ptr = ptr->next;
        }
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 12
Enter element 2: 23
Enter element 3: 34
Enter element 4: 45
Enter element 5: 56
1. Insert at beginning
2. Undo
3. Display list
Enter choice: 1
Enter element: 67
1. Insert at beginning
2. Undo
3. Display list
Enter choice: 3
Elements of list are 67 12      23      34      45      56
```

```
1. Insert at beginning
2. Undo
3. Display list
Enter choice: 2
1. Insert at beginning
2. Undo
3. Display list
Enter choice: 3
Elements of list are 12 23      34      45      56
```

```
1. Insert at beginning
2. Undo
3. Display list
Enter choice: 2
1. Insert at beginning
2. Undo
3. Display list
Enter choice: 3
Elements of list are 12 23      34      45      56
```

Experiment 3: Insert at End with Tail Optimization

// 3: Insert at end with tail optimisation

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
	int data;
	struct Node *next;
};
struct Node *insertAtEnd(struct Node *tail, int element);
void displayList(struct Node *head);
int main()
{
	int n, i, choice, element;
	struct Node *head, *tail, *newNode, *temp;
	printf("Enter number of nodes: ");
	scanf("%d", &n);
	if (n == 0)
	{
		head = tail = NULL;
	}
	else
	{
		newNode = (struct Node*)malloc(sizeof(struct Node));
		printf("Enter element 1: ");
		scanf("%d", &newNode->data);
		newNode->next = NULL;
		head = tail = newNode;
		temp = newNode;
		for (i = 2; i <= n; i++)
```

```
        {
                newNode = (struct Node*)malloc(sizeof(struct Node));
                printf("Enter element %d: ", i);
                scanf("%d", &newNode->data);
                newNode->next = NULL;
                tail = newNode;
                temp->next = newNode;
                temp = temp->next;
        }
}
do
{
        printf("1: Insert at end\n");
        printf("2: Display list\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
                case 1:
                        printf("Enter element: ");
                        scanf("%d", &element);
                        tail = insertAtEnd(tail, element);
                        break;
                case 2:
                        printf("Elements of list are ");
                        displayList(head);
                        printf("\n");
                        break;
```

```
                        default:
                                printf("Invalid choice\n");
                }
        } while (choice >= 1 && choice <= 2);
        return 0;
}
struct Node *insertAtEnd(struct Node *tail, int element)
{
        struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = element;
        newNode->next = NULL;
        tail->next = newNode;
        tail = newNode;
        return tail;
}
void displayList(struct Node *head)
{
        struct Node *ptr;
        ptr = head;
        while (ptr != NULL)
        {
                printf("%d\t", ptr->data);
                ptr = ptr->next;
        }
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 12
Enter element 2: 23
Enter element 3: 34
Enter element 4: 45
Enter element 5: 56
1: Insert at end
2: Display list
Enter choice: 1
Enter element: 67
1: Insert at end
2: Display list
Enter choice: 2
Elements of list are 12 23      34      45      56      67
```

Experiment 4: Position-Based Insertion and Deletion

```
// 4: Position based insertion and deletion

#include <stdio.h>

#include <stdlib.h>

struct Node

{

	int data;

	struct Node *next;

};

struct Node *insertAtPosition(struct Node *head, int element, int position);

struct Node *deleteByPosition(struct Node *head, int position);

void displayList(struct Node *head);

int main()

{

	int n, i, choice, element, position;

	struct Node *head, *newNode, *temp;

	printf("Enter number of nodes: ");

	scanf("%d", &n);

	if (n == 0)

	{

		head = NULL;

	}

	else
```

```
{
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element 1: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        head = newNode;
        temp = newNode;
        for (i = 2; i <= n; i++)
        {
                newNode = (struct Node*)malloc(sizeof(struct Node));
                printf("Enter element %d: ", i);
                scanf("%d", &newNode->data);
                newNode->next = NULL;
                temp->next = newNode;
                temp = temp->next;
        }
}
do
{
        printf("1: Insert at position\n");
        printf("2: Delete by position\n");
        printf("3: Display list\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
                case 1:
                        printf("Enter element: ");
```

```
                scanf("%d", &element);
                printf("Enter position: ");
                scanf("%d", &position);
                head = insertAtPosition(head, element, position);
                break;
            case 2:
                printf("Enter position: ");
                scanf("%d", &position);
                head = deleteByPosition(head, position);
                break;
            case 3:
                printf("Elements of linked list are ");
                displayList(head);
                printf("\n");
                break;
            default:
                printf("Invalid choice\n");
        }
    } while (choice >= 1 && choice <= 8);
    return 0;
}
struct Node *insertAtPosition(struct Node *head, int element, int position)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = element;
    int i;
    struct Node *fast, *slow;
```

```
        slow = fast = head;
        for (i = 0; i < position && fast != NULL; i++)
        {
                slow = fast;
                fast = fast->next;
        }
        if (slow == fast)
        {
                newNode->next = head;
                head = newNode;
                return head;
        }
        if (i == position)
        {
                slow->next = newNode;
                newNode->next = fast;
        }
        return head;
}
struct Node *deleteByPosition(struct Node *head, int position)
{
        if (head == NULL)
        {
                return NULL;
        }
        int i;
        struct Node *fast, *slow;
        slow = fast = head;
```

```
	for (i = 0; i < position && fast->next != NULL; i++)
	{
		slow = fast;
		fast = fast->next;
	}
	if (slow == fast)
	{
		head = fast->next;
		free(fast);
		return head;
	}
	if (i == position)
	{
		slow->next = fast->next;
		free(fast);
	}
	return head;
}
void displayList(struct Node *head)
{
	struct Node *ptr;
	ptr = head;
	while (ptr != NULL)
	{
		printf("%d\t", ptr->data);
		ptr = ptr->next;
	}
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 12
Enter element 2: 23
Enter element 3: 34
Enter element 4: 45
Enter element 5: 56
1: Insert at position
2: Delete by position
3: Display list
Enter choice: 1
Enter element: 67
Enter position: 4
1: Insert at position
2: Delete by position
3: Display list
Enter choice: 3
Elements of linked list are 12  23      34      45      67      56
```

```
1: Insert at position
2: Delete by position
3: Display list
Enter choice: 2
Enter position: 0
1: Insert at position
2: Delete by position
3: Display list
Enter choice: 3
Elements of linked list are 23  34      45      67      56
```

Experiment 5: Delete Nth Node from End (One Pass Only)

```
// 5: Delete nth node from end in one pass only

#include <stdio.h>

#include <stdlib.h>

struct Node

{

	int data;

	struct Node *next;

};

struct Node *deleteNthNodeFromEnd(struct Node *head, int n);

void displayList(struct Node *head);

int main()

{

	int n, i, choice, k;

	struct Node *head, *newNode, *temp;

	printf("Enter number of nodes: ");

	scanf("%d", &n);

	if (n == 0)

	{

		head = NULL;

	}

	else
```

```
{
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element 1: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        head = newNode;
        temp = newNode;
        for (i = 2; i <= n; i++)
        {
                newNode = (struct Node*)malloc(sizeof(struct Node));
                printf("Enter element %d: ", i);
                scanf("%d", &newNode->data);
                newNode->next = NULL;
                temp->next = newNode;
                temp = temp->next;
        }
}
do
{
        printf("1: Delete nth node from end\n");
        printf("2: Display list\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
                case 1:
                        printf("Enter position of node from end: ");
                        scanf("%d", &k);
```

```
                        head = deleteNthNodeFromEnd(head, k);
                        break;
                case 2:
                        printf("Elements of list are ");
                        displayList(head);
                        printf("\n");
                        break;
                default:
                        printf("Invalid choice\n");
            }
        } while (choice >= 1 && choice <= 2);
        return 0;
}
struct Node *deleteNthNodeFromEnd(struct Node *head, int n)
{
        struct Node *p, *q, *r;
        r = q = p = head;
        while (p->next != NULL && n != 0)
        {
            n--; // 3-1, 2-2, 1-3, 0-4
            p = p->next;
        }
        while (p->next != NULL && p != NULL)
        {
            r = q;
            q = q->next;
            p = p->next;
        }
```

```
        if (r == q)
        {
                head = q->next;
                free(q);
        }
        else
        {
                r->next = q->next;
                free(q);
        }
        return head;
}
void displayList(struct Node *head)
{
        struct Node *ptr;
        ptr = head;
        while (ptr != NULL)
        {
                printf("%d\t", ptr->data);
                ptr = ptr->next;
        }
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 12
Enter element 2: 23
Enter element 3: 34
Enter element 4: 45
Enter element 5: 56
1: Delete nth node from end
2: Display list
Enter choice: 1
Enter position of node from end: 4
1: Delete nth node from end
2: Display list
Enter choice: 2
Elements of list are 23 34      45      56
```

Experiment 6: Detect and Remove Cycle

```
// 6: Detect and remove cycle
#include <stdio.h>
#include <stdlib.h>
struct Node
{
	int data;
	struct Node *next;
};
int detectCycle(struct Node *head);
struct Node *detectCycleStart(struct Node *head);
void removeCycle(struct Node *head);
void displayList(struct Node *head);
int main()
{
	int n, i, choice;
	struct Node *head, *node1, *node2, *node3, *node4, *node5;
	node1 = (struct Node*)malloc(sizeof(struct Node));
	node2 = (struct Node*)malloc(sizeof(struct Node));
	node3 = (struct Node*)malloc(sizeof(struct Node));
	node4 = (struct Node*)malloc(sizeof(struct Node));
	node5 = (struct Node*)malloc(sizeof(struct Node));
	node1->data = 1;
	node1->next = node2;
	head = node1;
	node2->data = 2;
	node2->next = node3;
	node3->data = 3;
```

```
node3->next = node4;
node4->data = 4;
node4->next = node5;
node5->data = 5;
node5->next = node3;
do
{
        printf("1: Detect cycle\n");
        printf("2: Remove cycle\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
                case 1:
                        if (detectCycle(head))
                        {
                                printf("Linked list is cyclic\n");
                        }
                        else
                        {
                                printf("Linked list is not cyclic\n");
                        }
                        break;
                case 2:
                        removeCycle(head);
                        break;
                default:
                        printf("Invalid choice\n");
```

```
        }
    } while (choice >= 1 && choice <= 2);
    return 0;
}
int detectCycle(struct Node *head)
{
    struct Node *slow, *fast;
    slow = fast = head;
    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = (fast->next)->next;
        if (slow == fast)
        {
            return 1;
        }
    }
    return 0;
}
struct Node *detectCycleStart(struct Node *head)
{
    struct Node *slow, *fast;
    slow = fast = head;
    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = (fast->next)->next;
        if (slow == fast)
```

```
		{
			slow = head;
			while (slow != fast)
			{
				slow = slow->next;
				fast = fast->next;
			}
			return slow;
		}
	}
	return NULL;
}
void removeCycle(struct Node *head)
{
	struct Node *ptr, *start;
	ptr = head;
	start = detectCycleStart(head);
	if (start == NULL)
	{
		return;
	}
	while (ptr->next != start)
	{
		ptr = ptr->next;
	}
	ptr->next = NULL;
}
void displayList(struct Node *head)
```

```
{
	struct Node *ptr;
	ptr = head;
	while (ptr != NULL)
	{
		printf("%d\t", ptr->data);
		ptr = ptr->next;
	}
}
```

Output:

```
1: Detect cycle
2: Remove cycle
Enter choice: 1
Linked list is cyclic
1: Detect cycle
2: Remove cycle
Enter choice: 2
1: Detect cycle
2: Remove cycle
Enter choice: 1
Linked list is not cyclic
```

Experiment 7: Reverse Linked List in Groups of K

```
// 7: Reverse linked list in groups of k
#include <stdio.h>
#include <stdlib.h>
struct Node
{
	int data;
	struct Node *next;
};
struct Node *reverseListInGroups(struct Node *head, int k);
void displayList(struct Node *head);
int main()
{
```

```
int n, i, k;
struct Node *head, *newNode, *temp;
printf("Enter number of nodes: ");
scanf("%d", &n);
if (n == 0)
{
        head = NULL;
}
else
{
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element 1: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        head = newNode;
        temp = newNode;
        for (i = 2; i <= n; i++)
        {
                newNode = (struct Node*)malloc(sizeof(struct Node));
                printf("Enter element %d: ", i);
                scanf("%d", &newNode->data);
                newNode->next = NULL;
                temp->next = newNode;
                temp = temp->next;
        }
}
printf("Elements of linked list are ");
displayList(head);
```

```
	printf("\n");
	printf("Enter number of nodes in group: ");
	scanf("%d", &k);
	head = reverseListInGroups(head, k);
	printf("Elements of linked list are ");
	displayList(head);
	printf("\n");
	return 0;
}
struct Node *reverseListInGroups(struct Node *head, int k)
{
	int count;
	struct Node *ptr, *fast, *slow, *rev, *end, *temp;
	ptr = head;
	slow = fast = NULL;
	count = k;
	while (ptr != NULL && count != 0)
	{
		count--;
		fast = ptr;
		ptr = ptr->next;
		fast->next = slow;
		slow = fast;
	}
	rev = fast;
	while (ptr != NULL)
	{
		end = rev;
```

```
while (end->next != NULL)
{
        end = end->next;
}
slow = fast = NULL;
count = k;
while (ptr != NULL && count != 0)
{
        count--;
        fast = ptr;
        ptr = ptr->next;
        fast->next = slow;
        slow = fast;
}
temp = fast;
if (count != 0)
{
        ptr = fast;
        slow = fast = NULL;
        while (ptr != NULL)
        {
                fast = ptr;
                ptr = ptr->next;
                fast->next = slow;
                slow = fast;
        }
        temp = fast;
}
```

```
            end->next = temp;
        }
        return rev;
}
void displayList(struct Node *head)
{
        struct Node *ptr;
        ptr = head;
        while (ptr != NULL)
        {
            printf("%d\t", ptr->data);
            ptr = ptr->next;
        }
}
```

Output:

```
Enter number of nodes: 8
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Enter element 6: 6
Enter element 7: 7
Enter element 8: 8
Elements of linked list are 1   2       3       4       5       6       7       8
Enter number of nodes in group: 3
Elements of linked list are 3   2       1       6       5       4       7       8
```